Name: _____

CptS 355 Exam 3 (Fall 2026)

Write your name on every page. Keep explanations concise. For T/F questions, circle True or False. Snippets are independent; assume required C++ headers and enclosing functions are supplied. Rust excerpts are inside `main` unless shown otherwise. Assume the usual vtable implementation in C++ and safe Rust.

1. (7 pts) Give the output for the following Rust code. Which function uses generic static dispatch, and which uses a trait object for dynamic dispatch? Does the call `dynamic_value(&c)` require allocating `c` in a `Box` on the heap? Answer yes or no and briefly explain using the parameter type. Classify the declared interfaces, not possible compiler optimizations.

```
trait Value { fn value(&self) -> i32; }
struct Constant { n: i32 }
impl Value for Constant {
    fn value(&self) -> i32 { self.n }
}
fn static_value<T: Value>(x: &T) -> i32 { x.value() }
fn dynamic_value(x: &dyn Value) -> i32 { x.value() }
fn main() {
    let c = Constant { n: 7 };
    println!("{} {}", static_value(&c), dynamic_value(&c));
}
```

2. (4 pts) True or False: In the Scheme expression `(+ 2 3)`, `+` is the operator and `2` and `3` are its arguments.

3. (6 pts) Convert this infix expression into a valid Scheme S-expression, using only `+`, `-`, `*`, and `/`. Preserve the grouping and operand order; do not evaluate it or add a quotation mark:

$$(5 + 3) \cdot (9 - 2) - \frac{12}{4}$$

4. (6 pts) In Rust, read the supplied lifetime annotations; you do not need to write any. Which input is the returned reference tied to by `'a`: `x`, `y`, or both? Does the complete code compile, and if so what does it print? Does naming `'a` extend the lifetime of either owned string? Briefly explain.

```
fn first<'a>(x: &'a str, y: &str) -> &'a str {
    x
}
fn main() {
    let outer = String::from("tree");
    let result;
    {
        let inner = String::from("leaf");
        result = first(outer.as_str(), inner.as_str());
    }
    println!("{}", result);
}
```

5. (6 pts) Give the output for the following C++ code. Is `b` a separate object from `a`? Is `alias` a separate object or another name for `a`?

```
struct Counter {
    int n;
    Counter(int start) : n(start) {}
    void add(int amount) { n += amount; }
};
Counter a(2);
Counter b = a;
Counter& alias = a;
b.add(3);
alias.add(4);
std::cout << a.n << " " << b.n << " " << alias.n << "\n";
```

6. (6 pts) Consider these Rust excerpts independently, each inside `main`. For each, say whether it compiles. Give the output only for accepted code. Explain how the last use of `r` determines whether assigning to `n` is allowed.

```
// A
let mut n = 4;
let r = &n;
println!("{}", r);
n = 5;
println!("{}", n);
```

```
// B
let mut n = 4;
let r = &n;
n = 5;
println!("{}", r);
```

7. (4 pts) True or False: Under the usual Rust trait-object representation, a reference such as `&dyn T`, where `T` is a trait, contains both a data pointer and a vtable pointer.

8. (8 pts) Give the output for the following Rust code. What element type lets this vector refer to both implementing types? Does the vector own `c` and `s`, or borrow them? Must those two values remain alive while their references are used in the loop?

```
trait Value { fn value(&self) -> i32; }
struct Constant { n: i32 }
struct Square { side: i32 }
impl Value for Constant {
    fn value(&self) -> i32 { self.n }
}
impl Value for Square {
    fn value(&self) -> i32 { self.side * self.side }
}
fn main() {
    let c = Constant { n: 3 };
    let s = Square { side: 2 };
    let values: Vec<&dyn Value> = vec![&c, &s];
    for item in values { print!("{} ", item.value()); }
}
```

9. (5 pts) In Rust, `A` and `B` are empty structs. Consider the two calls below independently. Does each compile? Give the returned integer for an accepted call. Why is having a method named `value` not by itself enough to satisfy `T: Value`?

```
trait Value { fn value(&self) -> i32; }
struct A;
struct B;
impl Value for A {
    fn value(&self) -> i32 { 2 }
}
impl B {
    fn value(&self) -> i32 { 9 }
}
fn read<T: Value>(x: &T) -> i32 { x.value() }

// Call 1: read(&A)
// Call 2: read(&B)
```

10. (5 pts) In C++, is the returned pointer dangling solely because the local pointer variable `p` goes out of scope? Give the output if it is valid. Distinguish the lifetime of the pointer variable from the lifetime of the string literal's characters. The characters are never modified.

```
const char* label() {
    const char* p = "oak";
    return p;
}
const char* word = label();
std::cout << word << "\n";
```

11. (8 pts) In C++, complete three blanks in a recursive descent parser for this introductory S-expression grammar:

$$S \to \texttt{ATOM} \mid (\ I\ ) \qquad I \to SI \mid \epsilon$$

`ATOM` is one nonempty symbol or number token, not a parenthesis; ϵ denotes the empty string. The lexer has already separated parentheses from atoms. These helpers are supplied:

```
std::string peek(); // exact next token, or "" at end of input
std::string take(); // consume and return next token; error at end
bool isAtom(const std::string& token); // false for "", "(", ")"
void syntaxError(); // report an error; does not return
```

`Atom(text)` and the initially empty `List()` are supplied subclasses of `SExpr`. `List::append(child)` appends and owns a child. This function consumes one S; its caller checks for leftover tokens. Assume allocation succeeds; error-cleanup code is not required.

```
SExpr* parseSExpr() {
    if (isAtom(peek())) return new Atom(take());
    if (peek() != "(") syntaxError();
    take();
    List* result = new List();
    while (/* blank 1: the next token is not the closing token */) {
        if (peek() == "") syntaxError();
        result->append(/* blank 2: parse one child */);
    }
    /* blank 3: consume the closing token */
    return result;
}
```

Give the three replacements. Why does parsing `()` return an empty `List` without making a recursive call for a child?

12. (6 pts) In Rust, consider the two excerpts independently inside `main`, using this supplied type. Which excerpt compiles? Give the output only for accepted code. Explain the difference between the two receiver types `&mut self` and `&self`.

```
struct Counter { n: i32 }
impl Counter {
    fn inc(&mut self) { self.n += 1; }
    fn get(&self) -> i32 { self.n }
}

// A
let a = Counter { n: 2 };
a.inc();
println!("{}", a.get());

// B
let mut b = Counter { n: 2 };
b.inc();
println!("{}", b.get());
```

13. (4 pts) True or False: When a Rust `Box` owning an ordinary heap allocation is dropped normally, it drops its contained value and releases that allocation.

14. (3 pts) Give the output of this C++ templated function's two calls. Is the choice of the `int` versus `double` implementation made by compile-time template instantiation or run-time vtable dispatch?

```
template<class T>
T chooseLarger(T a, T b) { return a < b ? b : a; }
std::cout << chooseLarger(2, 9) << " "
          << chooseLarger(1.5, 0.5) << "\n";
```

15. (9 pts) In Rust, a `Box<Expr>` owns a child expression. Fill the blank so an `Add` evaluates its two children and adds their results. Give the output and show the two addition results. Does calling `eval` consume the expression, or can the caller evaluate it again? Why is no manual deletion needed for the boxed children when `main` finishes normally?

```
enum Expr {
    Num(i32),
    Add(Box<Expr>, Box<Expr>),
}
impl Expr {
    fn eval(&self) -> i32 {
        match self {
            Expr::Num(n) => *n,
            Expr::Add(left, right) => /* fill expression */,
        }
    }
}
fn main() {
    let expr = Expr::Add(
        Box::new(Expr::Num(4)),
        Box::new(Expr::Add(
            Box::new(Expr::Num(1)),
            Box::new(Expr::Num(2)))));
    println!("{}", expr.eval());
}
```

16. (4 pts) True or False: `Rc::clone(&p)` in Rust normally creates another owner of the existing allocation rather than a deep copy of the value inside it.

17. (6 pts) Give the output for the following Rust code. Could you also print `original` after the call without changing the existing statements or cloning the string? Explain briefly. Which binding owns the returned string at the final print? Is that returned value a dangling reference to the function's local binding?

```
fn add_mark(mut text: String) -> String {
    text.push('!');
    text
}
fn main() {
    let original = String::from("go");
    let result = add_mark(original);
    println!("{}", result);
}
```

18. (7 pts) Give the output for the following C++ code. In the first call, does the early `return` skip destruction of the already-constructed object `a`? Answer yes or no.

```
struct Mark {
    char name;
    Mark(char n) : name(n) { std::cout << "+" << name << " "; }
    ~Mark() { std::cout << "-" << name << " "; }
};
void run(bool stop) {
    Mark a('A');
    if (stop) return;
    Mark b('B');
}
run(true);
std::cout << "| ";
run(false);
```

19. (4 pts) True or False: A Scheme procedure that calls itself can still generate an iterative process, with no growing chain of deferred operations.

20. (4 pts) True or False: The parentheses delimiting an S-expression list are themselves additional atom elements of that list.

21. (4 pts) True or False: A value of a Rust `Copy` type may be assigned to a second binding while the original binding remains usable.

22. (4 pts) True or False: In Scheme, given `(define (square x) (* x x))` and `(define (use f) (f 4))`, the call `(use square)` passes the number 16 as the value of `f`.

Name: _____

CptS 355 Exam 3 (Fall 2026)

Write your name on every page. Keep explanations concise. For T/F questions, circle True or False. Snippets are independent; assume required C++ headers and enclosing functions are supplied. Rust excerpts are inside `main` unless shown otherwise. Assume the usual vtable implementation in C++ and safe Rust.

1. (7 pts) Give the output for the following Rust code. Which function uses generic static dispatch, and which uses a trait object for dynamic dispatch? Does the call `dynamic_value(&c)` require allocating `c` in a `Box` on the heap? Answer yes or no and briefly explain using the parameter type. Classify the declared interfaces, not possible compiler optimizations.

```
trait Value { fn value(&self) -> i32; }
struct Constant { n: i32 }
impl Value for Constant {
    fn value(&self) -> i32 { self.n }
}
fn static_value<T: Value>(x: &T) -> i32 { x.value() }
fn dynamic_value(x: &dyn Value) -> i32 { x.value() }
fn main() {
    let c = Constant { n: 7 };
    println!("{} {}", static_value(&c), dynamic_value(&c));
}
```

Solution / grading guide. The output is `7 7`. `static_value` uses generic static dispatch with a concrete instantiation for `Constant`. `dynamic_value` uses the `Value` trait object for dynamic dispatch through the appropriate method implementation.
No. Its parameter is `&dyn Value`, a borrowed trait object, not `Box<dyn Value>`. It can borrow the local `c` directly; dynamic dispatch does not itself require a heap allocation.
Award 1 point per printed value (2), 1 per dispatch classification (2), 1 for no, and 2 for identifying and explaining the borrowed parameter type.

2. (4 pts) True or False: In the Scheme expression `(+ 2 3)`, `+` is the operator and `2` and `3` are its arguments.

True

3. (6 pts) Convert this infix expression into a valid Scheme S-expression, using only `+`, `-`, `*`, and `/`. Preserve the grouping and operand order; do not evaluate it or add a quotation mark:

$$(5 + 3) \cdot (9 - 2) - \frac{12}{4}$$

Solution / grading guide.

```
(- (* (+ 5 3) (- 9 2)) (/ 12 4))
```

Award 1 point for outer subtraction, 2 for the product's two grouped operands, 2 for the quotient and its position as the right operand, and 1 for balanced prefix-call syntax and preserved operand order. Whitespace and line breaks are immaterial.

4. (6 pts) In Rust, read the supplied lifetime annotations; you do not need to write any. Which input is the returned reference tied to by `'a`: `x`, `y`, or both? Does the complete code compile, and if so what does it print? Does naming `'a` extend the lifetime of either owned string? Briefly explain.

```
fn first<'a>(x: &'a str, y: &str) -> &'a str {
    x
}
fn main() {
    let outer = String::from("tree");
    let result;
    {
        let inner = String::from("leaf");
        result = first(outer.as_str(), inner.as_str());
    }
    println!("{}", result);
}
```

Solution / grading guide. The returned reference is tied to `x`, not `y`. The code compiles and prints `tree`. The returned borrow points into `outer`, which is still alive at the final print; `inner` need not remain alive after the call.
No. Lifetime annotations describe constraints on valid borrows; they do not extend the lifetime of either owned string. The unused parameter warning for `y` is not a compilation failure.
Award 2 points for tying the return to `x` only, 1 for accepted, 1 for `tree`, and 2 for explaining that annotations describe borrow-validity constraints rather than extending the owners' lifetimes.

5. (6 pts) Give the output for the following C++ code. Is `b` a separate object from `a`? Is `alias` a separate object or another name for `a`?

```
struct Counter {
    int n;
    Counter(int start) : n(start) {}
    void add(int amount) { n += amount; }
};
Counter a(2);
Counter b = a;
Counter& alias = a;
b.add(3);
alias.add(4);
std::cout << a.n << " " << b.n << " " << alias.n << "\n";
```

Solution / grading guide. The output is `6 5 6`. `b` is a separate object whose integer member was copied from `a`. Changing `b.n` does not change `a.n`. `alias` is a reference to `a`, so `alias.add(4)` changes `a.n`.
Award 1 point per output value (3), 1 for the separate value copy, and 2 for identifying `alias` as another name for `a`. No additional explanation is required for those 2 points.

6. (6 pts) Consider these Rust excerpts independently, each inside `main`. For each, say whether it compiles. Give the output only for accepted code. Explain how the last use of `r` determines whether assigning to `n` is allowed.

```
// A
let mut n = 4;
let r = &n;
println!("{}", r);
n = 5;
println!("{}", n);
```

```
// B
let mut n = 4;
let r = &n;
n = 5;
println!("{}", r);
```

Solution / grading guide. A compiles and prints 4, then 5 on the next line. The shared borrow's last use is before the assignment. B is rejected: the assignment occurs while the shared borrow must remain valid for the later print of `r`. Do not predict output for B.
Award 1 point per classification (2), 2 for A's two output values, and 1 for explaining the borrow's last use in each excerpt (2). Do not require every borrow to last until the end of its lexical block.

7. (4 pts) True or False: Under the usual Rust trait-object representation, a reference such as `&dyn T`, where `T` is a trait, contains both a data pointer and a vtable pointer.

True

8. (8 pts) Give the output for the following Rust code. What element type lets this vector refer to both implementing types? Does the vector own `c` and `s`, or borrow them? Must those two values remain alive while their references are used in the loop?

```
trait Value { fn value(&self) -> i32; }
struct Constant { n: i32 }
struct Square { side: i32 }
impl Value for Constant {
    fn value(&self) -> i32 { self.n }
}
impl Value for Square {
    fn value(&self) -> i32 { self.side * self.side }
}
fn main() {
    let c = Constant { n: 3 };
    let s = Square { side: 2 };
    let values: Vec<&dyn Value> = vec![&c, &s];
    for item in values { print!("{} ", item.value()); }
}
```

Solution / grading guide. The output is `3 4`. The common element type is `&dyn Value`, a borrowed trait-object type. The vector contains references, not ownership of `c` and `s`. Both referents must remain alive for their uses, as they do here.
Award 1 point per output (2), 2 for the element type, 2 for borrowing rather than moving the referents, and 2 for validity through the loop's uses. Do not confuse the vector's own storage with ownership of the two referents. The trailing space does not affect grading.

9. (5 pts) In Rust, `A` and `B` are empty structs. Consider the two calls below independently. Does each compile? Give the returned integer for an accepted call. Why is having a method named `value` not by itself enough to satisfy `T: Value`?

```
trait Value { fn value(&self) -> i32; }
struct A;
struct B;
impl Value for A {
    fn value(&self) -> i32 { 2 }
}
impl B {
    fn value(&self) -> i32 { 9 }
}
fn read<T: Value>(x: &T) -> i32 { x.value() }

// Call 1: read(&A)
// Call 2: read(&B)
```

Solution / grading guide. `read(&A)` compiles and returns 2. `read(&B)` is rejected because `B` does not implement `Value`. `impl B` defines an inherent method, not an implementation of the trait. The bound requires an actual `Value` implementation, such as `impl Value for B`; method-name agreement alone does not supply it.
Award 1 point per classification (2), 1 for the returned 2, and 2 for distinguishing an inherent method from an explicit trait implementation.

10. (5 pts) In C++, is the returned pointer dangling solely because the local pointer variable `p` goes out of scope? Give the output if it is valid. Distinguish the lifetime of the pointer variable from the lifetime of the string literal's characters. The characters are never modified.

```
const char* label() {
    const char* p = "oak";
    return p;
}
const char* word = label();
std::cout << word << "\n";
```

Solution / grading guide. No. The code is valid and prints `oak`. The pointer value is copied out of the function; it is not the address of the local pointer variable. The string literal's characters have static storage duration and remain alive after the call. Losing a local pointer binding does not by itself destroy the pointed-to data.
Award 1 point for no, 1 for output, 1 for the copied pointer value, and 2 for distinguishing local binding from static-duration character data.

11. (8 pts) In C++, complete three blanks in a recursive descent parser for this introductory S-expression grammar:

$$S \to \texttt{ATOM} \mid (\ I\ ) \qquad I \to SI \mid \epsilon$$

`ATOM` is one nonempty symbol or number token, not a parenthesis; ϵ denotes the empty string. The lexer has already separated parentheses from atoms. These helpers are supplied:

```
std::string peek(); // exact next token, or "" at end of input
std::string take(); // consume and return next token; error at end
bool isAtom(const std::string& token); // false for "", "(", ")"
void syntaxError(); // report an error; does not return
```

`Atom(text)` and the initially empty `List()` are supplied subclasses of `SExpr`. `List::append(child)` appends and owns a child. This function consumes one S; its caller checks for leftover tokens. Assume allocation succeeds; error-cleanup code is not required.

```
SExpr* parseSExpr() {
    if (isAtom(peek())) return new Atom(take());
    if (peek() != "(") syntaxError();
    take();
    List* result = new List();
    while (/* blank 1: the next token is not the closing token */) {
        if (peek() == "") syntaxError();
        result->append(/* blank 2: parse one child */);
    }
    /* blank 3: consume the closing token */
    return result;
}
```

Give the three replacements. Why does parsing `()` return an empty `List` without making a recursive call for a child?

Solution / grading guide. Blank 1: `peek() != ")"`. Blank 2: `parseSExpr()`. Blank 3: `take();`.
After the opening parenthesis of `()` is consumed, the next token is already `")"`. The loop runs zero times, so the list stays empty. The final `take()` consumes that closing token. The explicit end-of-input test in the loop rejects an unclosed list.
Award 2 points per replacement (6), and 2 for explaining the zero-iteration loop on the empty list and consumption of its closing token. Equivalent code using the supplied helpers is acceptable; do not require an evaluator, lexer, or error-cleanup code.

12. (6 pts) In Rust, consider the two excerpts independently inside `main`, using this supplied type. Which excerpt compiles? Give the output only for accepted code. Explain the difference between the two receiver types `&mut self` and `&self`.

```
struct Counter { n: i32 }
impl Counter {
    fn inc(&mut self) { self.n += 1; }
    fn get(&self) -> i32 { self.n }
}

// A
let a = Counter { n: 2 };
a.inc();
println!("{}", a.get());

// B
let mut b = Counter { n: 2 };
b.inc();
println!("{}", b.get());
```

Solution / grading guide. A is rejected: its immutable binding cannot be mutably borrowed for `inc`. B compiles and prints 3. The `inc` receiver is an exclusive mutable borrow permitting its field update. The `get` receiver is a shared borrow for reading; neither receiver takes ownership.
Award 1 point per classification (2), 1 for output, 2 for the mutable receiver/binding requirement, and 1 for the shared reading receiver.

13. (4 pts) True or False: When a Rust `Box` owning an ordinary heap allocation is dropped normally, it drops its contained value and releases that allocation.

True

14. (3 pts) Give the output of this C++ templated function's two calls. Is the choice of the `int` versus `double` implementation made by compile-time template instantiation or run-time vtable dispatch?

```
template<class T>
T chooseLarger(T a, T b) { return a < b ? b : a; }
std::cout << chooseLarger(2, 9) << " "
          << chooseLarger(1.5, 0.5) << "\n";
```

Solution / grading guide. The output is `9 1.5`. The compiler instantiates the templated function for `int` and `double`; no virtual method table is involved in selecting these implementations.
Award 1 point per result (2) and 1 for compile-time template instantiation. No explanation beyond the requested classification is required.

15. (9 pts) In Rust, a `Box<Expr>` owns a child expression. Fill the blank so an `Add` evaluates its two children and adds their results. Give the output and show the two addition results. Does calling `eval` consume the expression, or can the caller evaluate it again? Why is no manual deletion needed for the boxed children when `main` finishes normally?

```
enum Expr {
    Num(i32),
    Add(Box<Expr>, Box<Expr>),
}
impl Expr {
    fn eval(&self) -> i32 {
        match self {
            Expr::Num(n) => *n,
            Expr::Add(left, right) => /* fill expression */,
        }
    }
}
fn main() {
    let expr = Expr::Add(
        Box::new(Expr::Num(4)),
        Box::new(Expr::Add(
            Box::new(Expr::Num(1)),
            Box::new(Expr::Num(2)))));
    println!("{}", expr.eval());
}
```

Solution / grading guide. The blank is `left.eval() + right.eval()`. The inner sum is $1 + 2 = 3$, and the root computes $4 + 3 = 7$, which is printed. `&self` borrows rather than consumes the expression, so another evaluation is allowed. Dropping the owned root drops its fields; each owned `Box` drops its child and releases its allocation recursively.
Award 2 points for the expression, 2 for the two sums/output, 1 for borrowing, 1 for permitting reevaluation, 2 for recursively dropping owned children, and 1 for automatic release without manual deletion.

16. (4 pts) True or False: `Rc::clone(&p)` in Rust normally creates another owner of the existing allocation rather than a deep copy of the value inside it.

True

17. (6 pts) Give the output for the following Rust code. Could you also print `original` after the call without changing the existing statements or cloning the string? Explain briefly. Which binding owns the returned string at the final print? Is that returned value a dangling reference to the function's local binding?

```
fn add_mark(mut text: String) -> String {
    text.push('!');
    text
}
fn main() {
    let original = String::from("go");
    let result = add_mark(original);
    println!("{}", result);
}
```

Solution / grading guide. The output is `go!`. No: ownership moves from `original` into the parameter, so the original binding cannot be used afterward. The function returns an owned `String`, moved into `result`; it is not returning a reference to a destroyed local value.
Award 1 point for output, 1 for no, 2 for explaining the argument move, 1 for `result` as owner, and 1 for distinguishing an owned return from a dangling reference.

18. (7 pts) Give the output for the following C++ code. In the first call, does the early `return` skip destruction of the already-constructed object `a`? Answer yes or no.

```
struct Mark {
    char name;
    Mark(char n) : name(n) { std::cout << "+" << name << " "; }
    ~Mark() { std::cout << "-" << name << " "; }
};
void run(bool stop) {
    Mark a('A');
    if (stop) return;
    Mark b('B');
}
run(true);
std::cout << "| ";
run(false);
```

Solution / grading guide. The output is `+A -A | +A +B -B -A`. No: returning from the function destroys its constructed automatic objects. In the first call, `b` is never constructed. In the second, `b` is destroyed before `a`.
Award 3 points for the first call's trace, 3 for the second call's construction/destruction order, and 1 for no. The trailing space does not affect grading.

19. (4 pts) True or False: A Scheme procedure that calls itself can still generate an iterative process, with no growing chain of deferred operations.

True

20. (4 pts) True or False: The parentheses delimiting an S-expression list are themselves additional atom elements of that list.

False

21. (4 pts) True or False: A value of a Rust `Copy` type may be assigned to a second binding while the original binding remains usable.

True

22. (4 pts) True or False: In Scheme, given `(define (square x) (* x x))` and `(define (use f) (f 4))`, the call `(use square)` passes the number 16 as the value of `f`.

False